김민재

minjae2k1@gmail.com
01059331625 | 2001년생

단순히 AI를 활용하는 사람과 'AI를 활용하는 개발자'는 무엇이 달라야 할까요?

그 격차를 만들어나가는 방법으로서 저는 'Core형 개발자'와 'PO형 개발자'를 지향합니다.

AI agent의 작업 결과는 단순히 주어진 문제를 해결하는 하나의 방법이어서는 안됩니다.
알맞은 tradeoff를 설정하고 이슈 발생 시 대처 방안을 고려하는 등 장기적인 관리 가능성을 보장해야 합니다.
이와 같이 시스템의 핵심을 안정적으로 구축하고 유지하는 것이 제가 생각하는 'Core형 개발자'의 역할입니다.

동시에, AI가 놓치기 쉬운 특수한 비즈니스 맥락을 반영할 수 있어야 합니다.
이를 위해서는 빠르고 깊이 있는 방식으로 도메인에 대한 이해를 지속적으로 갖출 필요가 있습니다.
덧붙여, 본인의 영역뿐만 아니라 프로덕트 전반의 요소를 파악하고 있어야 합니다.
이를 바탕으로 주어진 과제를 기술적으로 해결하는 데에만 그치지 않고 문제를 주도적으로 해결해나가는 것이 제가 생각하는 'PO형 개발자'의 역할입니다.

개발자로서의 지향점과는 별개로, 현실과 가상이 융합된 초거대 규모 상호작용 플랫폼에 흥미를 갖고 있습니다.
단순한 오락이나 임시 회의 공간에 그치는 것이 아니라, 또 하나의 라이프스타일 공간을 만들고 가꾸어 나가는 과정에 기여해보고 싶습니다.

개발 직무 서버/백엔드 개발자 · SW/솔루션

기술 스택 C · C++ · Git · Visual Studio Code · Kotlin · IntelliJ IDEA · Spring Framework

학력

2026.08 졸업 예정 — 대학교(4년) | **한국과학기술원**
전산학부
3.69 / 4.3

2020.02 졸업 — 고등학교 | **북일고등학교**

경력(업무경험)

개발경력 0년 6개월 (총 경력 1년 3개월)

2025.09 - 2026.02
0년 6개월

토스뱅크

국내 인터넷은행 중 하나로, '토스' 앱 내에서 서비스를 제공하고 있습니다.

여신엔지니어링플랫폼팀 / Server Developer Intern

Kotlin | Spring Framework | IntelliJ IDEA | MySQL | MongoDB | Kafka | Redis

1. MSA 환경에서 공인전자문서센터 게이트웨이 서비스 구축
- 대출 과정에서 생기는 여러 서류를 공인전자문서센터에 저장
- 사내 요청을 처리하는 '오케스트레이터' 계층과 각 공인전자문서센터와의 상호작용을 처리하는 '벤더' 계층을 분리하고 Kafka를 활용하여 연결
- ShedLock 및 코루틴을 통해 인스턴스 간 충돌을 방지하면서 충분한 처리량을 보장하는 스케줄러 설계 및 개발
- 외부 API timeout 및 사내 인프라 K8S pod의 lifecycle 정책 등을 고려하여 스케줄러의 Graceful termination 구현
- 보다 방어적인 대처로서, 공인전자문서센터에 전자문서 저장하는 절차 중 셧다운 발생 시 자동으로 절차 재개할 수 있는 방안 수립
- 처리 지연을 비롯한 운영 중 이슈 발생 시 Flighting 등을 활용하여 재배포 없이 대응할 수 있는 방안 수립
- AutoCloseable 구현체로 임시 파일을 관리하여 리소스 관리

2. 특정 데이터 누락된 요청 감지 및 경보 기능 개발
- Http 요청 헤더에 특정 값이 없는 경우 요청이 수행된 IP로 마이크로서비스를 추정한 후 해당 서비스의 담당자에게 메시지 발송
- HttpServletRequest의 remoteAddr, 프록시 환경에서의 'X-Forwarded-For'를 활용하여 요청이 수행된 IP 획득
- Batchjob으로 사내 시스템으로부터 서비스 별 IP 데이터를 조회 후 MongoDB에 저장해두고, 특정 IP에 해당하는 마이크로서비스 추정
- 특정 서비스의 담당자를 조회하는 API 서버에 가해지는 부하를 줄이기 위하여 Caffeine을 활용한 로컬 캐싱 및 Resilience4J의 Bulkhead를 활용한 동시 요청 수행량 조절

2022.01 - 2022.09
0년 9개월

(주)엘엠엔틱바이오텍 비개발

DGIST nBEST 연구실의 원천 기술을 기반으로 자기장을 활용한 혈액 속 종양세포 탐지 기술을 연구하는 체외 진단 의료기기 스타트업입니다.

Co-Founder (SW)

Trello | Microsoft Office 365

1. 사업화 방향성 수립 및 투자 유치 위한 자료 제작
- 2022년 4월, 대구경북기술지주로부터 2천만 원 규모 투자 유치
- 2022년 6월, TIPS 선정되어 5억 원 규모 투자 유치
2. Trello를 비롯한 협업 방식 도입
3. LabView로 개발된 실험장비 제어 코드 분석
- 자기장 발생을 위한 '모터 파트' 분석
- 데이터 측정을 위한 '센서 파트' 분석
4. 의료기기 제어 목적의 소프트웨어를 위한 기능명세서 작성
- 자기장 발생, 자기장 회전 등 장비가 수행하는 동작을 블록화
- 블록을 배치하여 사용자가 원하는 동작 순서를 설정
- 카메라 제어, 목표 마킹 등 모니터링 관련 기능 구체화

프로젝트

2023.01 - 2023.01

(주)스텔스솔루션 '주소 변이 서비스'

공격자가 서비스 서버를 발견할 수 없도록 해당 서버에 장착된 랜카드의 IP주소를 주기적으로 변경하는 솔루션입니다.

C++ Ubuntu Visual Studio Code

(주)투에프씨에서 단기로 업무를 수행하였습니다.
Ubuntu 환경에서 동작하는 서비스 프로세스 대상의 콘솔 제어 프로그램 개발을 담당했습니다.
1. 서비스와의 쌍방향 데이터 교환 목적으로 Shared memory에 기반한 IPC API 구현
- 데이터 분할 및 자체 정의한 객체 형태로 변환
- 공유 메모리 내에 배치된 큐를 통한 객체 I/O 구현
2. CLI 환경에서의 서비스 프로세스 설정 확인 및 수정 기능 구현
3. Linux 명령어를 활용한 기능 구현
- 'xinetd'를 활용하여 서비스 프로세스의 Telnet 파트 제어
- 'tail'을 활용한 실시간 로그 모니터링

2021.01 - 2021.02

(주)프라코 아산공장 MES(통합관리시스템)

공장 내 설비 관리, 부품 및 생산물 대상의 재고 관리, 생산 과정 실시간 모니터링 등 공장에서 진행되는 전 과정을 전산화한 통합관리시스템입니다. 기존에 사용 중이던 MES를 바탕으로 개발되었습니다.

C# MSSQL Windows Visual Studio

(주)투에프씨에서 단기로 업무를 수행하였습니다.
'설비관리' 파트를 담당했으며, 기존에 고객사에서 사용 중이던 소프트웨어 및 설계된 사항에 기반하여 개발 진행하였습니다.
1. C# Winform을 활용하여 UI 제작 및 내부 로직 구현

- DB에 데이터 요청하여 표 형식으로 시각화
- 버튼, 텍스트박스 등에 대응하는 이벤트 핸들러 구현
2. MS-SQL 프로시저 작성
- SQL로 작성된 프로시저를 클라이언트에서 API처럼 호출 가능

기타사항

TOEIC

어학 | ETS

2021년 8월 : 합계 955점 (LC: 480, RC: 475)
2026년 1월 : 합계 855점 (LC: 385, RC: 470)

TOPCIT (소프트웨어 역량 검정)

자격증 | 정보통신기획평가원

2025년 5월 : 합계 724점 (1000점 만점)
- 기술 영역 : 464점 (565점 만점)
- 비즈니스 영역 : 90점 (175점 만점)
- 통합 영역 : 170점 (260점 만점)
- 수준 '4' (문제해결형) : 기술 및 비즈니스 영역에 대한 지식과 스킬을 응용하여 과제를 해결할 수 있는 수준

자기소개서

대학교 강의와 연계된 프로젝트 경험 (pintos)

2022년 12월부터 2023년 4월까지, 전역 후 2024년 12월부터 2025년 1월까지, 교육 목적의 미니 OS를 구현해보는 프로젝트인 'pintos'를 진행했습니다. 이는 KAIST의 '운영체제 및 실험' 과목에서 수행하는 프로젝트입니다. 단독으로 진행했으며, 다음은 이 프로젝트에서 제가 내렸던 대표적인 기술적 선택입니다:

① 삽입 정렬을 통한 프로세스 큐 재정렬: 실행 준비가 된 프로세스 중에서 CPU가 다음에 실행할 프로세스를 결정할 때, 또는 락(lock) 등을 획득하기 위해 블록된 상태로 대기 중이던 프로세스 중 하나를 언블록할 때, 후보 프로세스 중 가장 높은 우선순위를 가진 프로세스가 선택되어야 합니다. 이를 저는 우선순위를 기준으로 정렬된 큐에서 pop하는 방식으로 구현했습니다. 이때 큐에 새로운 프로세스를 넣거나 큐에 있는 프로세스의 우선순위가 변하는 등 큐를 재정렬해야하는 경우 해당 프로세스를 제외하면 이미 정렬된 상태임에 주목하여, 삽입 정렬이 효율적인 방식이라 판단했습니다.

② 비트맵을 활용한 Disk sector의 Lazy allocation 구현: 파일 시스템을 구현할 때 각 파일마다 적절한 분량의 디스크 섹터를 할당해야 했습니다. 이때 효율적인 할당 정책으로, 모든 비트가 0인 부분에는 당장 할당하지 않고 이후 내용이 작성된 뒤에 섹터가 할당되도록 구현했습니다. 이를 위해 파일 안 각 영역의

섹터 할당 여부를 파일의 inode에 기록해야했습니다. 연속된 영역을 이분법적으로 다루는 점에 착안하여, 저는 비트맵이 아이노드가 과하게 커지지 않는 최선의 방법이라 판단했습니다. 실제로도 아이노드 용량의 약 2000배에 달하는 영역을 표현할 수 있었습니다.

그외에도 시스템 콜, Page fault 대처와 Swap In/Out 등 OS에 포함된 여러 추상화를 구현하는 과정에서 저는 단순히 구현하는 것을 넘어 속도와 안정성의 균형점을 찾아야 했습니다. 예를 들어 타이머 인터럽트 핸들러에서 발생하는 연산을 가능한 한 빠르게 만들기 위해 중간 계산값을 메모리에 저장하는 방식으로 구현했습니다. 이러한 경험은 실제 서비스를 개발할 때 겪는 상황과 유사점이 많다고 생각이 들었습니다. 좋은 구조를 체험하고 여러 기술적 선택을 내리는 과정에서 저는 개발자로서 크게 성장할 수 있었습니다.

대학교 강의와 연계된 프로젝트 경험 (TodoGenie)

2022년 9월부터 같은 해 12월까지, 단어의 사용 빈도를 바탕으로 사용자가 할 일을 등록할 때 단어 추천을 해주는 크로스 플랫폼 투두리스트 서비스인 '투두지니'를 진행했습니다. 이는 KAIST '소프트웨어 공학 개론' 과목에서 수행한 3인 협동 캡스톤 프로젝트였습니다. 저는 프론트엔드, 테스팅, PM을 담당했습니다.

저는 변경이 발생한 컴포넌트만 재렌더링하여 사용자의 동작에 빠르게 반응한다는 장점을 가진 React를 기반으로 개발을 진행했습니다. UI 제작 측면에서, 저는 로그아웃 버튼, 도움말 버튼 등 사용자가 보는 대부분의 화면에서 변경 없이 유지되는 부분을 파악한 뒤 별도의 컴포넌트로 구현하는 등의 방식으로 해당 장점을 살리려 했습니다. 한편, 이 프로젝트는 웹 앱, 모바일 앱의 클라이언트와 클라우드 서버가 API를 통해 소통하는 구조로, 다양한 환경에서 이용이 가능해야 했습니다. 특히 웹 앱의 경우 접속 기기 종류가 많은 만큼 어떤 크기의 화면에서도 UI가 제대로 표시되는 것이 중요했습니다. 이에 미디어쿼리를 활용하여 화면 비율에 따라 2개의 UI를 선택적으로 표시하는 방안을 고려했습니다. 하지만 스택 형식으로 위에서 아래로 층을 쌓는 구조의 UI라면 PC와 모바일 환경 모두에서 제대로 내용물이 표시되는 걸 확인했고, 개발의 편의성을 위하여 하나의 UI를 상대 스케일 활용 및 최솟값·최댓값 지정을 통해 크기만 조정하여 표시하는 방안을 채택했습니다.

당시 테스팅 과목을 수강하고 있었기에 테스팅 시나리오를 주도적으로 작성했습니다. 우선, 웹 앱, 모바일 앱, 서버 각각에 대한 테스팅 목표를 설정했습니다. 이후 팀 내에서 한 명씩 다른 인원이 개발한 파트를 맡아 팀 내 테스팅을 거친 뒤, 테스터 역할을 수행하는 다른 팀에게 서비스 사용 시나리오를 전달하여 사용자 대상 테스팅을 진행했습니다. 테스팅 시나리오를 작성할 때 일반적인 Use-case 외의 예외적인 사례까지 포함되도록 특히 주의했습니다. 버튼 클릭, 텍스트 입력 등 각 화면에서 가능한 동작을 대상으로, 해당 동작의 순서를 바꾸거나 여러 번 반복 수행하는 등의 방법으로 가능한 한 많은 테스팅 시나리오를 작성하여 테스트 커버리지를 최대한 넓히고자 했습니다. 이를 통해 사용자 대상 테스팅에서 웹 앱과 모바일 앱의 비밀번호 최소 길이 정책이 다른 문제를 비롯해 총 27개의 문제점을 식별할 수 있었습니다.

그외에도 창업 경험을 바탕으로, 팀 리더 및 프로젝트 매니저(PM) 역할을 수행했습니다. 폭포수 모델을 적용하여 요구사항 분석, 세부 설계, 구현, 테스팅의 4단계로 개발을 진행했습니다. 각 단계에서 Task를 분할하여 효율적인 작업 배분을 시도했고, 평균적으로 각 Task는 계획보다 7.9일이 지연되었습니다.

스타트업에서 근무할 당시 제품 개발을 겪어보지 못해 다소 아쉬운 점이 있었습니다. 하지만 이 강의를 듣고 실제로 프로젝트도 진행하는 과정에서, 단순히 같이 일하는 것이 아닌, 제품 개발을 위하여 전략적

으로 협업하는 방법을 경험할 수 있었습니다. 또한 개발자는 기술적인 역량 뿐만 아니라 커뮤니케이션 능력 등 팀 내에서 협업하는 역량 또한 갖춰야 함을 다시금 깨닫는 계기가 되었습니다.

링크

Github	https://github.com/1026minjae
포트폴리오	https://polydactyl-spice-70c.notion.site/Minjae-Kim-1d90d7e5d704806faa47e13bc856d